

Duração 2h. Tol. 30 minutos

Sem consulta

1. [12,5%] O código abaixo representa dois conceitos bem conhecidos. Analise com atenção o código e indique quais as linhas onde há erros de compilação, ou, sem serem de compilação ainda assim causam comportamentos imprevisíveis. Para cada um justifique o erro em questão e aponte uma possível correção.

```
01 #include <iostream>
02 using namespace std;
03
04 class Ponto {
05     int x,y;
06     Ponto(int x, int y) { x = x; y = y; } {}
07     int getX() const { return x; }
08     int getY() const { return y; }
09     void setX(int pos) const { x = pos; }
10     void setY(int pos) const { y = pos; }
11 };
12
13 struct Figura {
14     const string tipo;
15     const Ponto origem;
16     float area;
17     Figura(string t, Ponto p, float a) {
18         tipo = t;
19         origem = p;
20         area = a;
21     }
22     Figura() {}
23     virtual ~Figura() {}
24     ostream & operator << (ostream & saida) { /* omitido */ }
25 };
26
27 main() {
28     Ponto a(1,2);
29     Figura f("pentagono", a, 30);
30     Figura g;
31     f = g;
32     cout << g;
33 }
34 }
```

2. [12,5%] Analise o código apresentado abaixo. Indique as mensagens que o programa escreve, o texto exato pela ordem exata.

```
#include <iostream>
using namespace std;

class A {
public:
    A() { cout << "A default constructor\n"; }
    A(int v) {
        this->v = v;
        cout << "A constructor v=" << v << "\n";
    }
    virtual ~A() { cout << "~A destructor v=" << v << "\n"; }
    void printValue() { cout << "v=" << v << "\n"; }
    int v=0;
};

class B : public A {
public:
    B() : A(5) { cout << "B default constructor\n"; }
    B(int v) {
        this->v = v;
        cout << "B constructor v=" << v << "\n";
    }
    B& operator=(int v) {
        this->v = v;
        cout << "B= v=" << v << "\n";
        return *this;
    }
    ~B() { cout << "~B destructor v=" << v << "\n"; }
    void printValue() { cout << "v=" << (v*2) << "\n"; }
};

int main() {
    cout << "-- start --\n";
    B b1(1);
    cout << "-----\n";
    { B b2 = 2; }
    cout << "-----\n";
    B b3;
    b3 = 3;
    cout << "-----\n";
    A a;
    cout << "-----\n";
    A* p = new B(4);
    p->printValue();
    delete p;
    cout << "-- end --\n";
    return 0;
}
```

3. [25%] Considere um autocarro. Trata-se de um veículo de transporte público, partilhando as características habituais destes veículos: pode estar parado, ou em movimento, ou está a circular ou está parado numa determinada paragem, inicia movimento, trava, admite passageiros que tenham bilhete válido, deixa sair passageiros, etc..

Os autocarros têm um número de lugares sentados que varia obrigatoriamente entre 20 e 60, definido no ato da sua construção, não sendo possível viajar de pé. Os lugares são identificados por um número de 1 a N (N é o número de lugares). Os passageiros são representados pelos seus bilhetes que identificam o número de lugar e o nome da estação destino.

Cada autocarro tem uma rota que consiste num conjunto de paragens, cada uma representada pelo seu nome. O conjunto de paragens (a rota) é especificado no ato de criação do autocarro. A rota pode também ser (re)definida a qualquer instante desde que o autocarro não tenha passageiros.

O passageiro (representado pelo seu bilhete) só é admitido se o seu bilhete indicar uma estação que faça parte da rota do autocarro e se o lugar indicado estiver livre. Sempre que o autocarro chega a uma estação todos os lugares ocupados por passageiros com destino a essa estação ficam livres. Adicionalmente, um passageiro pode sair se tal for pedido explicitamente (indicando o número do lugar). Chegar a uma estação corresponde a estar parado e ser invocado um mecanismo do autocarro que recebe o nome da estação que faz parte do seu percurso.

- a) Escreva a classe Autocarro que suporte o comportamento descrito, incluindo outras funcionalidades que possam ser necessárias. Pode acrescentar mais código / classes se achar que é útil.

- b) Adicione à classe Autocarro a funcionalidade para eliminar da rota do autocarro todas as estações que contenham uma determinada palavra que é indicada. Nesta alínea **deve obrigatoriamente** usar as funcionalidades disponíveis quando faz **#include <algorithm>**.

4. [30%] Uma agência imobiliária pretende um sistema para armazenar informações acerca dos imóveis que possui para venda e das respetivas ofertas de compra. Uma venda representa um determinado imóvel que está disponível no mercado. O conjunto das vendas pertence à imobiliária, é partilhado por todos os seus mediadores da agência, e não é possível duplicá-la (a venda), por motivos de estratégia comercial.

Foram desenvolvidas as seguintes classes para representar: **Venda** (que inclui a informação sobre o imóvel a que diz respeito a venda e a lista de ofertas feitas para o comprar), **Oferta** (que corresponde à oferta de compra submetida por um possível comprador), e **Imovel** (que representa a propriedade em processo de venda). Os comentários ajudam a perceber a lógica relativa ao controlo dos dados por parte dos objetos.

```

class Venda {                                     // representa um imóvel para
venda
    Imovel* imovel;                               // imóvel já existente que agora entrou no
mercado
    std::vector<Oferta*> ofertas;                   // ofertas para esta Venda
(pertencem-lhe)
};

class Oferta {                                     // representa uma pessoa que talvez queira comprar
    string nome;
    int valorOferecido;
public:
    Oferta(const std::string& nome, const int& valorOferecido);
    Oferta(const Oferta& outraOferta);
    const std::string& obtemNome() const;           // nome da pessoa
    const int& obtemOferta() const;                 // valor oferecido
};

class Imovel {                                     // Imóvel, independentemente de estar para venda ou não
    int valor;
public:
    Imovel(const int& valor);
    const int& obtemValor() const;
};

```

Reescreva a classe Venda, e implemente os métodos que permitam a funcionalidade indicada abaixo e os métodos que possam ser necessários ao bom funcionamento dos seus objetos. Pode acrescentar coisas à classe Venda mas não pode mudar o que já lá está. Nos exemplos seguintes, venda é um objecto da classe Venda.

- Criar vendas, sendo necessário indicar apenas o imóvel, não existindo ofertas inicialmente;
- Introduzir uma nova oferta (interessado em comprar) à venda com <<


```
Exemplo: Oferta o{"Carlos Henriques", 125000};
venda << o;
```
- Remover da lista de ofertas todas aquelas cujo valor oferecido seja inferior ao um determinado valor, recorrendo a -=


```
Exemplo: venda -= 135000;
```
- Remover o interessado cuja oferta é a mais baixa através de --.


```
Exemplo: --venda;
```
- Aceder às informações constantes dos compradores indicando o seu nome e usando []


```
Exemplo: venda["José Costa"]->obtemOferta();
```
- Ao terminar, cada venda deve eliminar todas as informações cuja posse seja exclusiva do objeto. O imóvel tem uma existência independente de estar ou não para venda.

Importante:

- Na sua resposta não deve especificar #includes e não deve separar o código em .h e .cpp.
- Não é permitida a utilização de *auto* na resposta a esta questão.

5. [20%] Pretende-se simular a operação de um aeroporto em C++. Existem diferentes tipos de aeronaves (tais como jatos particulares, aviões militares, etc.), mas para este exame em particular, deve-se focar em dois: **aviões de passageiros** e **cargueiros**. Todas as aeronaves têm uma matrícula, um nº de horas totais de voo, nº de horas de voo desde a última manutenção, peso em toneladas, nº máximo de litros de combustível que suporta e indicador de nível do combustível. Um avião de passageiros tem também associado a informação sobre o nº de passageiros que consegue transportar. Um cargueiro além dos dados que todas as aeronaves possuem, tem também associado a quantidade em toneladas que consegue transportar.

A cada **pouso**, o nº total de horas de voo da aeronave é atualizado, sendo o aeroporto informado acerca da duração do voo que pouso (nº de horas) e o número de quilómetros até ao destino seguinte. Ao pousarem, devem ocorrer nas aeronaves operações de manutenção e de abastecimento, consoante o tipo de aeronave.

Implemente as **classes** e **funcionalidades** que aparecem a **negrito** no texto abaixo. Assuma que todas as aeronaves operam na sua capacidade máxima (aviões de passageiros voam cheios e cargueiros transportam a totalidade da sua capacidade).

Um **aeroporto recebe** aeronaves (corresponde a aeronave que **pousam**):

- **Avião de passageiros:** Ao fim de mais de 100 horas de voo desde a última manutenção completa é feita uma nova **manutenção** completa em que todos os parâmetros são avaliados pelos técnicos, o que corresponde ao contador de horas, desde a última manutenção ser reiniciado. Quanto ao **abastecimento**, o depósito é abastecido com a quantidade de combustível dada por:

$$1,1 * \text{nº de quilómetros} * (\text{número de passageiros} * 8 / 100 + \text{peso da aeronave}).$$

- **Cargueiros:** caso o voo tenha tido uma duração superior a 10 horas é feita uma manutenção completa; caso contrário, só é feita uma manutenção completa após totalizar mais de 50 horas de voo acumuladas desde a última manutenção. Independentemente da duração do voo e das horas acumuladas desde a última manutenção, é sempre feita uma verificação adicional a cada 500 horas de voos, medidas desde a entrada em serviço da aeronave (corresponde a imprimir a mensagem “verificação de segurança”). Quanto ao **abastecimento**, corresponde a colocar no depósito a quantidade de combustível dado por:

$$\text{nº de quilómetros} * 2 * (\text{carga} + \text{peso da aeronave}).$$

As aeronaves que pousam permanecem no aeroporto até descolarem para a viagem seguinte. Fazer **descolar** tem como efeito remover (sem a destruir) a aeronave do aeroporto identificada pela sua matrícula. As aeronaves já foram informadas acerca do número de quilómetros para o destino seguinte quando pousaram e não é preciso nada de especial em termos de informação para a fazer descolar.

Os aviões não pertencem ao aeroporto mas sim às respectivas companhias. No entanto, **se o aeroporto for destruído**, os aviões atualmente no aeroporto são naturalmente destruídos também.